

Numerical Methods Lab

Lab 19 – 26 | Python Programs

Python Source Code

Lab-19: Jacobi Iteration Method

Question: Write a program for Jacobi Iteration Method. $4x + y + 2z = 4$, $3x + 5y + z = 7$, $x + y + 3z = 3$

Python Program:

```
def get_coefficients():
    eqs = []
    for i in range(3):
        print(f"\nEquation {i+1}:")
        a = float(input(f" Coefficient of x (a{i+1}): "))
        b = float(input(f" Coefficient of y (b{i+1}): "))
        c = float(input(f" Coefficient of z (c{i+1}): "))
        d = float(input(f" Constant term (d{i+1}): "))
        eqs.append([a, b, c, d])
    return eqs

def jacobi(tol=1e-6, max_iter=100):
    print("Enter the coefficients for the system of equations (ax + by + cz = d):")
    eq = get_coefficients()
    x, y, z = 0.0, 0.0, 0.0
    print(f"\n{'Iter':<5} | {'x':<10} | {'y':<10} | {'z':<10} | {'Max Error':<15}")
    print("-" * 65)
    for i in range(1, max_iter + 1):
        x_new = (eq[0][3] - (eq[0][1]*y + eq[0][2]*z)) / eq[0][0]
        y_new = (eq[1][3] - (eq[1][0]*x + eq[1][2]*z)) / eq[1][1]
        z_new = (eq[2][3] - (eq[2][0]*x + eq[2][1]*y)) / eq[2][2]
        error = max(abs(x_new-x), abs(y_new-y), abs(z_new-z))
        x, y, z = x_new, y_new, z_new
        print(f"{'i':<5} | {'x':<10.6f} | {'y':<10.6f} | {'z':<10.6f} | {'error':<15.6e}")
        if error < tol:
            print(f"\nConverged in {i} iterations.")
            print(f"Final Solution: x={x:.6f}, y={y:.6f}, z={z:.6f}")
            return x, y, z
        print("\nDid not converge within max iterations.")
    return x, y, z

jacobi()
```

Lab-20: Gauss-Seidel Method

Question: Write a program for Gauss-Seidel Method. $4x + y + 2z = 4$, $3x + 5y + z = 7$, $x + y + 3z = 3$

Python Program:

```
def get_coefficients():
    eqs = []
    for i in range(3):
        print(f"\nEquation {i+1}:")
        a = float(input(f" Coefficient of x (a{i+1}): "))
        b = float(input(f" Coefficient of y (b{i+1}): "))
        c = float(input(f" Coefficient of z (c{i+1}): "))
        d = float(input(f" Constant term (d{i+1}): "))
        eqs.append([a, b, c, d])
    return eqs

def gauss_seidel(tol=1e-6, max_iter=100):
    print("Enter the coefficients for the system (ax + by + cz = d):")
    eq = get_coefficients()
    x, y, z = 0.0, 0.0, 0.0
    print(f"\n{'Iter':<5} | {'x':<10} | {'y':<10} | {'z':<10} | {'Max Error':<15}")
    print("-" * 65)
    for i in range(1, max_iter + 1):
        x_old, y_old, z_old = x, y, z
        x = (eq[0][3] - (eq[0][1]*y + eq[0][2]*z)) / eq[0][0]
        y = (eq[1][3] - (eq[1][0]*x + eq[1][2]*z)) / eq[1][1]
        z = (eq[2][3] - (eq[2][0]*x + eq[2][1]*y)) / eq[2][2]
        error = max(abs(x-x_old), abs(y-y_old), abs(z-z_old))
        print(f"{i:<5} | {x:<10.6f} | {y:<10.6f} | {z:<10.6f} | {error:<15.6e}")
        if error < tol:
            print(f"\nConverged in {i} iterations.")
            print(f"Final Solution: x={x:.6f}, y={y:.6f}, z={z:.6f}")
            return x, y, z
    print("\nDid not converge.")
    return x, y, z

gauss_seidel()
```

Lab-21: Taylor Series Method

Question: Write a program for Taylor series expansion. $dy/dx = 2y/x$, $x_0=4$, $y_0=6$, $h=0.25$, evaluate at $x=6$

Python Program:

```
import math

def f(x, y):
    return 2 * (y / x)

def taylor_method():
    print("Taylor Series Method for dy/dx = 2y/x")
    print("-" * 40)
    x0 = float(input("Enter initial x (x0): "))
    y0 = float(input("Enter initial y (y0): "))
    xn = float(input("Enter target x value: "))
    h = float(input("Enter step size (h): "))
    x, y = x0, y0
    steps = int(round((xn - x0) / h))
    print(f"\n{'Step':<5} | {'x':<10} | {'y':<10}")
    print("-" * 30)
    print(f"{0:<5} | {x:<10.4f} | {y:<10.4f}")
    for i in range(1, steps + 1):
        f1 = 2 * y / x
        f2 = 2 * (x * f1 - y) / (x * x)
        f3 = 2 * (x * x * f2 - 2 * x * f1 + 2 * y) / (x ** 3)
        y = y + h*f1 + (h**2/2)*f2 + (h**3/6)*f3
        x = round(x + h, 10)
        print(f"{i:<5} | {x:<10.4f} | {y:<10.4f}")
        print("-" * 30)
    print(f"Approximate solution at x = {xn} is y = {y:.6f}")

taylor_method()
```

Lab-22: Euler's Method

Question: Write a program for Euler's method. $dy/dx = 2y/x$, $x_0=4$, $y_0=6$, $h=0.25$, evaluate at $x=6$

Python Program:

```
def f(x, y):
    return 2 * (y / x)

def euler_method():
    print("Euler's Method Solver for dy/dx = 2y/x")
    print("-" * 40)
    x0 = float(input("Enter initial x (x0): "))
    y0 = float(input("Enter initial y (y0): "))
    xn = float(input("Enter target x value to solve for: "))
    h = float(input("Enter step size (h): "))
    x, y = x0, y0
    steps = int(round((xn - x0) / h))
    print(f"\n{'Step':<5} | {'x':<10} | {'y':<10}")
    print("-" * 30)
    print(f"{0:<5} | {x:<10.4f} | {y:<10.4f}")
    for i in range(1, steps + 1):
        y = y + h * f(x, y)
        x = round(x + h, 10)
        print(f"{i:<5} | {x:<10.4f} | {y:<10.4f}")
    print("-" * 30)
    print(f"Approximate solution at x = {xn} is y = {y:.6f}")

euler_method()
```

Lab-23: Heun's Method

Question: Write a program for Heun's method. $dy/dx = x + y$, $x_0=0$, $y_0=1$, $h=0.1$, evaluate at $x=1$

Python Program:

```
def f(x, y):
    return x + y

def heuns_method():
    print("Heun's Method Solver for dy/dx = x + y")
    print("-" * 45)
    x0 = float(input("Enter initial x (x0): "))
    y0 = float(input("Enter initial y (y0): "))
    xn = float(input("Enter target x value: "))
    h = float(input("Enter step size (h): "))
    x, y = x0, y0
    steps = int(round((xn - x0) / h))
    print(f"\n{'Step':<5} | {'x':<10} | {'y':<10} | {'Slope Avg':<10}")
    print("-" * 50)
    print(f"{'0':<5} | {'x':<10.4f} | {'y':<10.4f} | {'N/A':<10}")
    for i in range(1, steps + 1):
        k1 = f(x, y)
        y_pred = y + h * k1
        x_next = x + h
        k2 = f(x_next, y_pred)
        slope_avg = (k1 + k2) / 2
        y = y + h * slope_avg
        x = x_next
    print(f"{'i':<5} | {'x':<10.4f} | {'y':<10.4f} | {'slope_avg':<10.4f}")
    print("-" * 50)
    print(f"Approximate solution at x = {xn} is y = {y:.6f}")

heuns_method()
```

Lab-24: Runge-Kutta 4th Order (RK4) Method

Question: Write a program for RK4 method. $dy/dx = 3x + y$, $x_0=2$, $y_0=4$, $h=0.1$, evaluate at $x=3$

Python Program:

```
def f(x, y):
    return 3 * x + y

def rk4_method():
    print("RK4 Method Solver for dy/dx = 3x + y")
    print("-" * 50)
    x0 = float(input("Enter initial x (x0): "))
    y0 = float(input("Enter initial y (y0): "))
    xn = float(input("Enter target x value: "))
    h = float(input("Enter step size (h): "))
    x, y = x0, y0
    steps = int(round(abs(xn - x0) / h))
    print(f"\n{'Step':<5} | {'x':<10} | {'y':<10}")
    print("-" * 35)
    print(f"{'0':<5} | {'x':<10.4f} | {'y':<10.4f}")
    for i in range(1, steps + 1):
        k1 = h * f(x, y)
        k2 = h * f(x + h/2, y + k1/2)
        k3 = h * f(x + h/2, y + k2/2)
        k4 = h * f(x + h, y + k3)
        y = y + (k1 + 2*k2 + 2*k3 + k4) / 6
        x = x + h
    print(f"{'i':<5} | {'x':<10.4f} | {'y':<10.4f}")
    print("-" * 35)
    print(f"Approximate solution at x = {xn} is y = {y:.6f}")

rk4_method()
```

Lab-25: Laplace Equation

Question: Write a program for Laplace equation using finite difference method.

Python Program:

```
MAX = 100
TOL = 1e-6

def initialize(grid, n):
    for i in range(n):
        grid[0][i] = 1.0
        grid[n-1][i] = 0.0
        grid[i][0] = 0.0
        grid[i][n-1] = 1.0

def laplace(grid, n):
    error = float('inf')
    while error > TOL:
        error = 0.0
        for i in range(1, n-1):
            for j in range(1, n-1):
                old = grid[i][j]
                grid[i][j] = 0.25 * (grid[i-1][j] + grid[i+1][j] + grid[i][j-1] + grid[i][j+1])
                error = max(error, abs(grid[i][j] - old))

def print_grid(grid, n):
    for i in range(n):
        for j in range(n):
            print(f"{grid[i][j]:8.4f}", end=" ")
        print()

n = int(input("Enter the size of the grid (n x n): "))
grid = [[0.0] * MAX for _ in range(MAX)]
initialize(grid, n)
laplace(grid, n)
print_grid(grid, n)
```

Lab-26: Poisson Equation

Question: Write a program for Poisson equation using finite difference method.

Python Program:

```
def f(x, y):
    return 2 * x * x * y * y

def initialize(n):
    return [[0.0] * n for _ in range(n)]

def poisson(grid, n, left, right, bottom, upper, acc):
    for i in range(n):
        grid[0][i] = upper
        grid[n-1][i] = bottom
        grid[i][0] = left
        grid[i][n-1] = right
        error = float('inf')
        while error > acc:
            error = 0.0
            for i in range(1, n-1):
                for j in range(1, n-1):
                    old = grid[i][j]
                    grid[i][j] = 0.25 * (grid[i-1][j] + grid[i+1][j] +
                                         grid[i][j-1] + grid[i][j+1] -
                                         f(i/(n-1), j/(n-1)))
                    error = max(error, abs(grid[i][j] - old))

def print_grid(grid):
    for row in grid:
        print(" ".join(f"{v:8.4f}" for v in row))

length = float(input("Enter length of the plate: "))
width = float(input("Enter width of the plate: "))
n = int(input("Enter the size of the grid: "))
left, right, bottom, upper = map(float, input("Enter boundary temperatures (left, right, bottom, upper): ").split())
acc = float(input("Enter accuracy limit: "))
grid = initialize(n)
poisson(grid, n, left, right, bottom, upper, acc)
print(f"\nDimensions: Length = {length:.2f}, Width = {width:.2f}")
print("Solution (Temperatures):")
print_grid(grid)
```